

Name of the Student: **KALUVALA SRI SAI PRAJWALA**

Roll NO: **B23CS137**

College: **KITSW**

## 1. Project Title

**Brain Tumor MRI Classification using Deep Learning**

## 2. IDE Code

Write:

The IDE code is implemented in Google Colab using Python. It includes environment setup, dataset acquisition from Kaggle, data preprocessing, exploratory data analysis, image normalization, data augmentation, and preparation of training, validation, and testing datasets. Model implementation is intentionally excluded as per the current project milestone.

**Open the GOOGLE COLAB FILE :**

[https://colab.research.google.com/drive/1pf0sf3ZqkdkdMLip93gEGJ\\_goM436VvY?usp=sharing](https://colab.research.google.com/drive/1pf0sf3ZqkdkdMLip93gEGJ_goM436VvY?usp=sharing)

## 3. Algorithm

### Step-by-Step Procedure

1. Import the required Python libraries.
2. Download the Brain Tumor MRI dataset from Kaggle.
3. Load the dataset into the Colab environment.
4. Organize the dataset into Training and Testing folders.
5. Explore the dataset by checking the number of images in each class.
6. Display sample MRI images.
7. Resize images to **128 × 128** pixels.
8. Normalize pixel values to the range **0–1**.
9. Apply image augmentation techniques.
10. Create training, validation, and testing generators.
11. Prepare the processed data for deep learning model implementation.

### Logic / Method Used

The system preprocesses MRI images before classification. The preprocessing includes resizing, normalization, and data augmentation to improve image quality and prepare a balanced dataset. The processed dataset is then divided into training, validation, and testing sets, making it ready for CNN-based classification in the next phase.

### Model / Technique Applied

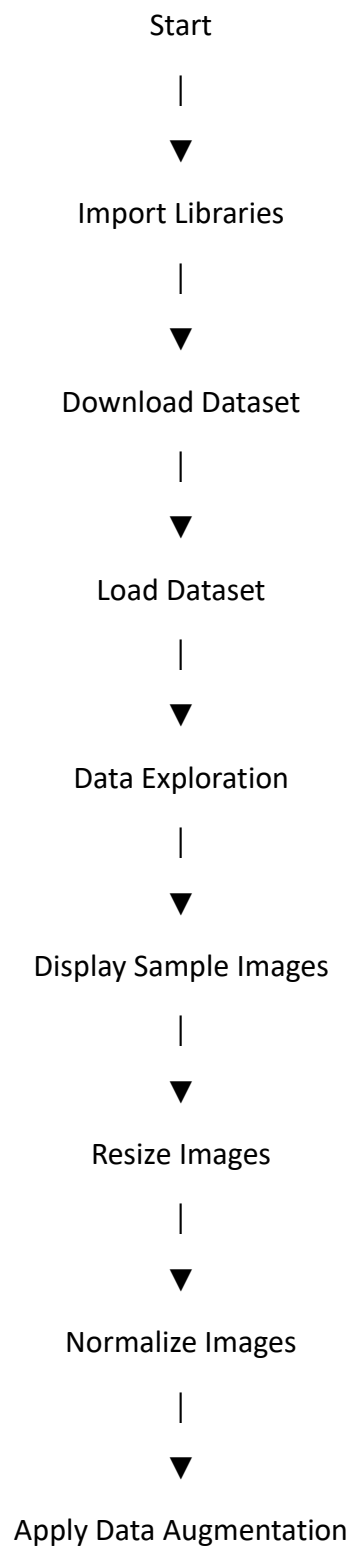
## Proposed Model

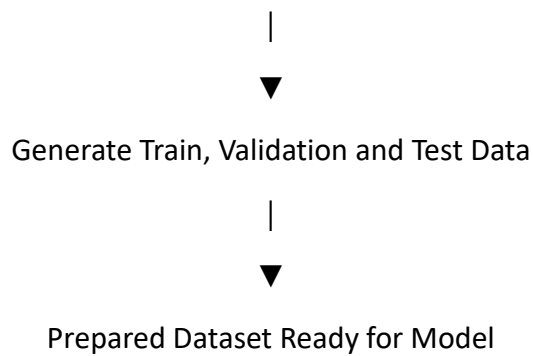
- Convolutional Neural Network (CNN)
- Transfer Learning using MobileNetV2

## Current Progress

Only dataset preparation and preprocessing have been completed. Model implementation and training will be performed in the next milestone.

### Process Flow





#### 4. Dataset

##### Dataset Name

Brain Tumor MRI Dataset

##### Source

Kaggle

Dataset Link

<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>

##### Number of Samples

##### Dataset Images

Training 5712

Testing 1311

Total 7023

##### Features / Attributes

- MRI Brain Image
- Tumor Label

##### Classes

- Glioma
- Meningioma
- Pituitary
- No Tumor

##### Training and Testing Split

- Training Images
- Validation Images (20% of Training Data)

- Testing Images

## Data Format

- JPG Images
- RGB Color Images
- Resized to **128 × 128 × 3**

## 5. IDE Code Snippet & Results

```
print("=" * 88)
print("SECTION 1: ENVIRONMENT SETUP")
print("=" * 88)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_curve, auc,
    roc_auc_score, accuracy_score, precision_recall_fscore_support
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize
import cv2
import os
import warnings
from tqdm.notebook import tqdm
import time
from collections import Counter

# Deep Learning
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.callbacks import (
    EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
)
from tensorflow.keras.utils import to_categorical

warnings.filterwarnings('ignore')

# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Configure plotting style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
```

```
from google.colab import files

print("=" * 88)
print("SECTION 2: DATASET DOWNLOAD & EXTRACTION")
print("=" * 88)

# Install Kagglehub (only first time)
!pip -q install kagglehub

import kagglehub
import os
import shutil

print("Downloading Brain Tumor MRI Dataset from Kaggle...")
print()

# Download dataset automatically
dataset_path = kagglehub.dataset_download(
    "masoudnickpervar/brain-tumor-mri-dataset"
)

print("\ Dataset downloaded successfully!")
print(f"Location: {dataset_path}")

# Create the same folder structure expected by the rest of the notebook
destination = "brain_tumor_data"

if not os.path.exists(destination):
    shutil.copytree(dataset_path, destination)

print()
print("\ Dataset ready!")
print(f"Training Folder :", os.path.join(destination, "Training"))
print(f"Testing Folder  :", os.path.join(destination, "Testing"))
print()
```

```
=====
SECTION 2: DATASET DOWNLOAD & EXTRACTION
=====
```

UNLOCKING BRAIN TUMOR PRE-PROCESSING FROM KAGGLE...

Using Colab cache for faster access to the 'brain-tumor-mri-dataset'  
✓ Dataset downloaded successfully!  
Location: /kaggle/input/brain-tumor-mri-dataset

✓ Dataset ready!  
Training Folder : brain\_tumor\_data/Training  
Testing Folder : brain\_tumor\_data/Testing

Medical imaging preprocessing is crucial for model performance. According to standardization across different MRI scanners and protocols requires:

1. Intensity normalization (z-score or min-max)
2. Skull stripping (when working with raw DICOM)
3. Registration to common anatomical template
4. Resampling to consistent resolution

```
DATA_DIR = 'brain_tumor_data'
TRAIN_DIR = os.path.join(DATA_DIR, 'Training')
TEST_DIR = os.path.join(DATA_DIR, 'Testing')

# Image parameters
IMG_HEIGHT = 128
IMG_WIDTH = 128
BATCH_SIZE = 32
NUM_CLASSES = 4
CLASS_NAMES = ['glioma', 'meningioma', 'notumor', 'pituitary']
```

## EXPLORATORY DATA ANALYSIS

Understanding the dataset is critical before modeling. We need to:

1. Check class distribution (potential imbalance)
2. Analyze image dimensions and quality
3. Visualize sample images from each class
4. Identify preprocessing requirements

```
def count_images_per_class(data_dir):
    """Count images in each class directory"""
    class_counts = {}
    for class_name in os.listdir(data_dir):
        class_path = os.path.join(data_dir, class_name)
        if os.path.isdir(class_path):
            count = len([f for f in os.listdir(class_path)
                        if f.endswith(('.jpg', '.png', '.jpeg'))])
            class_counts[class_name] = count
    return class_counts

train_counts = count_images_per_class(TRAIN_DIR)
test_counts = count_images_per_class(TEST_DIR)

print("Training Set Distribution:")
for class_name, count in sorted(train_counts.items()):
    print(f" {class_name:15s}: {count:4d} images")
print(f" {'TOTAL':15s}: {sum(train_counts.values()):4d} images")
print()

print("Testing Set Distribution:")
for class_name, count in sorted(test_counts.items()):
    print(f" {class_name:15s}: {count:4d} images")
print(f" {'TOTAL':15s}: {sum(test_counts.values()):4d} images")
print()

# Visualize class distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

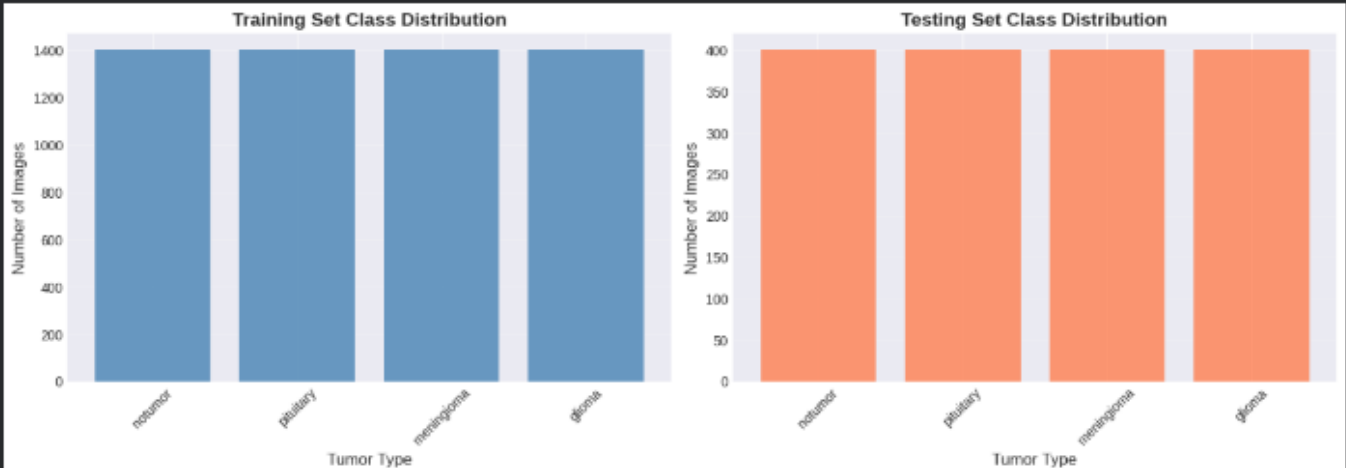
# Training distribution
axes[0].bar(train_counts.keys(), train_counts.values(), color='steelblue', alpha=0.8)
axes[0].set_title('Training Set Class Distribution', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Tumor Type', fontsize=12)
axes[0].set_ylabel('Number of Images', fontsize=12)
axes[0].tick_params(axis='x', rotation=45)
axes[0].grid(axis='y', alpha=0.3)

# Testing distribution
axes[1].bar(test_counts.keys(), test_counts.values(), color='coral', alpha=0.8)
axes[1].set_title('Testing Set Class Distribution', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Tumor Type', fontsize=12)
axes[1].set_ylabel('Number of Images', fontsize=12)
```

```
print("\n Observation: Check if classes are balanced or if we need class weights")
print()
```

```
*** Training Set Distribution:
glioma      : 1400 images
meningioma  : 1400 images
notumor     : 1400 images
pituitary   : 1400 images
TOTAL       : 5600 images
```

```
Testing Set Distribution:
glioma      : 400 images
meningioma  : 400 images
notumor     : 400 images
pituitary   : 400 images
TOTAL       : 1600 images
```



```

print("Setting up data augmentation pipeline...")

# Training data generator with augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,           # Normalize to [0,1]
    rotation_range=15,        # Random rotation +15°
    width_shift_range=0.1,    # Horizontal shift
    height_shift_range=0.1,   # Vertical shift
    shear_range=0.1,          # Shear transformation
    zoom_range=0.15,          # Random zoom
    horizontal_flip=True,     # Random horizontal flip
    fill_mode='constant',     # Fill with zeros
    cval=0,                   # Fill value
    validation_split=0.15     # 15% for validation
)

# Test data generator (only rescaling, no augmentation)
test_datagen = ImageDataGenerator(rescale=1./255)

# Load training data with augmentation
train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    color_mode='grayscale',
    shuffle=True,
    subset='training',
    seed=42
)

# Load validation data (from training split, no augmentation in validation)
validation_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    color_mode='grayscale',
    shuffle=False,
    subset='validation',
    seed=42
)

```

```

# Load test data
test_generator = test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    color_mode='grayscale',
    shuffle=False
)

print(f"✓ Training samples: {train_generator.samples}")
print(f"✓ Validation samples: {validation_generator.samples}")
print(f"✓ Test samples: {test_generator.samples}")
print(f"✓ Class indices: {train_generator.class_indices}")
print()
print("Visualizing augmentation effects...")

sample_batch = next(iter(train_generator))
original_image = sample_batch[0][0] # First image from batch

# Generate augmented versions
augmented_images = []
temp_gen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode='constant'
)

# Create multiple augmented versions
img_array = np.expand_dims(original_image, 0)
aug_iter = temp_gen.flow(img_array * 255, batch_size=1) # Rescale back
for i in range(8):
    aug_img = next(aug_iter)[0].astype('uint8')
    augmented_images.append(aug_img)

# Plot original and augmented
fig, axes = plt.subplots(3, 3, figsize=(12, 12))
fig.suptitle('Data Augmentation Examples', fontsize=16, fontweight='bold')

```

```

# Create multiple augmented versions
img_array = np.expand_dims(original_image, 0)
aug_iter = temp_gen.flow(img_array * 255, batch_size=1) # Rescale back
for i in range(8):
    aug_img = next(aug_iter)[0].astype('uint8')
    augmented_images.append(aug_img)

# Plot original and augmented
fig, axes = plt.subplots(3, 3, figsize=(12, 12))
fig.suptitle('Data Augmentation Examples', fontsize=16, fontweight='bold')

axes[0, 0].imshow(original_image.squeeze(), cmap='gray')
axes[0, 0].set_title('Original Image', fontweight='bold')
axes[0, 0].axis('off')

for idx, aug_img in enumerate(augmented_images):
    row = (idx + 1) // 3
    col = (idx + 1) % 3
    axes[row, col].imshow(aug_img.squeeze(), cmap='gray')
    axes[row, col].set_title(f'Augmented {idx+1}')
    axes[row, col].axis('off')

plt.tight_layout()
plt.savefig('augmentation_examples.png', dpi=100, bbox_inches='tight')
plt.show()

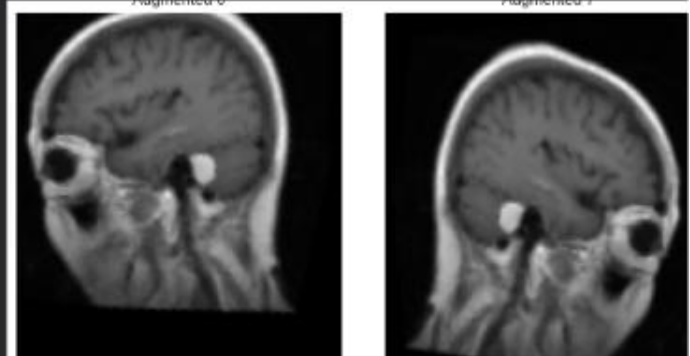
print("✓ Augmentation visualization complete")
print()

# Compute class weights for handling imbalance
print("⚠ Computing class weights for imbalanced data...")

class_counts_array = np.array([train_counts[cn] for cn in sorted(CLASS_NAMES)])
total_samples = np.sum(class_counts_array)
class_weights = total_samples / (NUM_CLASSES + class_counts_array)
class_weight_dict = {i: weight for i, weight in enumerate(class_weights)}

print("Class weights:")
for i, (class_name, weight) in enumerate(zip(sorted(CLASS_NAMES), class_weights)):
    print(f"  {class_name:15s}: {weight:.4f}")
print()

```



✓ Augmentation visualization complete

⚠ Computing class weights for imbalanced data...

Class weights:

|            |         |
|------------|---------|
| glioma     | : 1.000 |
| meningioma | : 1.000 |
| notumor    | : 1.000 |
| pituitary  | : 1.000 |

# Display sample images from each class

```

def load_sample_images(data_dir, samples_per_class=3):
    """Load sample images from each class"""
    samples = {}
    for class_name in os.listdir(data_dir):
        class_path = os.path.join(data_dir, class_name)
        if os.path.isdir(class_path):
            image_files = [f for f in os.listdir(class_path)
                           if f.endswith(('.jpg', '.png', '.jpeg'))]
            sample_files = np.random.choice(image_files,
                                             min(samples_per_class, len(image_files)),
                                             replace=False)
            samples[class_name] = sample_files
    return samples

```



```

        images = []
        for img_file in sample_files:
            img_path = os.path.join(class_path, img_file)
            img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
            images.append(img)
        samples[class_name] = images
    return samples

sample_images = load_sample_images(TRAIN_DIR, samples_per_class=4)

# Plot sample images
fig, axes = plt.subplots(4, 4, figsize=(15, 15))
fig.suptitle('Sample MRI Images from Each Class', fontsize=16, fontweight='bold', y=0.995)

for idx, (class_name, images) in enumerate(sorted(sample_images.items())):
    for i, img in enumerate(images):
        axes[idx, i].imshow(img, cmap='gray')
        axes[idx, i].set_title(f'{class_name}\n{img.shape}', fontsize=10)
        axes[idx, i].axis('off')

    # Add simple statistics
    mean_intensity = np.mean(img)
    std_intensity = np.std(img)
    axes[idx, i].text(0.5, -0.15, f'μ={mean_intensity:.1f}, σ={std_intensity:.1f}',
                    transform=axes[idx, i].transAxes,
                    ha='center', fontsize=8, color='darkblue')

plt.tight_layout()
plt.savefig('sample_images.png', dpi=300, bbox_inches='tight')
plt.show()

print("\n Key Observations:")
print(" - Image dimensions vary (need resizing)")
print(" - Intensity ranges differ (need normalization)")
print(" - Different contrast levels (augmentation will help)")
print()

def analyze_image_dimensions(data_dir):
    """Analyze distribution of image dimensions"""
    dimensions = {}
    for class_name in os.listdir(data_dir):
        class_path = os.path.join(data_dir, class_name)

```

```

        class_path = os.path.join(data_dir, class_name)
        if os.path.isdir(class_path):
            for img_file in os.listdir(class_path):
                if img_file.endswith(('.jpg', '.png', '.jpeg')):
                    img_path = os.path.join(class_path, img_file)
                    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
                    dimensions.append(img.shape)

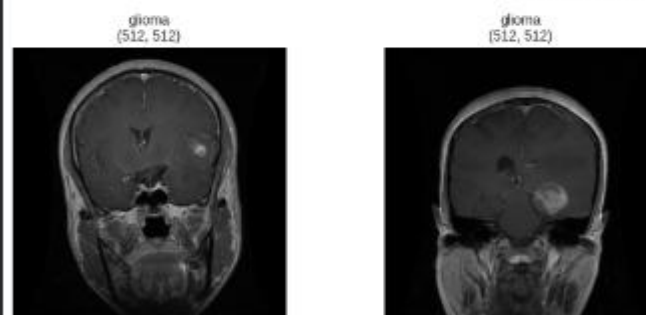
    return dimensions

print("\n Analyzing image dimensions (sampling 200 images)...")
all_dims = analyze_image_dimensions(TRAIN_DIR)
sampled_dims = np.random.choice(len(all_dims), min(200, len(all_dims)), replace=False)
unique_dims = set([all_dims[i] for i in sampled_dims])

print(f" Found {len(unique_dims)} unique dimensions (from sample)")
print(f" Most common dimensions:")
dim_counter = Counter(all_dims)
for dim, count in dim_counter.most_common(5):
    print(f" {dim}: {count} images")
print(f" + Standardizing to {IMG_HEIGHT}x{IMG_WIDTH} for consistent processing")
print()

```

Sample MRI Image



```

# Key Observations:
- Image dimensions vary (need resizing)
- Intensity ranges differ (need normalization)
- Different contrast levels (augmentation will help)

# Analyzing image dimensions (sampling 200 images)...
Found 42 unique dimensions (from sample)
Most common dimensions:
(512, 512): 4812 images
(225, 225): 253 images
(630, 630): 69 images
(251, 281): 48 images
(236, 236): 39 images
+ Standardizing to 128x128 for consistent processing

# =====
# TRANSFER LEARNING DATA PIPELINE (RGB + MobileNetV2 preprocess)
# =====

from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

print("\n Setting up TRANSFER LEARNING generators (RGB + preprocess_input)...")

# IMPORTANT:
# - Do NOT use rescale=1./255 when using preprocess_input for MobileNetV2.
# - preprocess_input expects raw pixels in [0, 255] and normalizes internally.

train_datagen_t1 = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode='constant',
    cval=0,
    validation_split=0.15
)

test_datagen_t1 = ImageDataGenerator(preprocessing_function=preprocess_input)

```

```

train_generator_t1 = train_datagen_t1.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    color_mode='rgb', # <-- key difference vs baseline
    shuffle=True,
    subset='training',
    seed=1
)

validation_generator_t1 = train_datagen_t1.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    color_mode='rgb',
    shuffle=False,
    subset='validation',
    seed=1
)

test_generator_t1 = test_datagen_t1.flow_from_directory(
    TEST_DIR,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    color_mode='rgb',
    shuffle=False
)

print(f"✓ T1 Training samples: {train_generator_t1.samples}")
print(f"✓ T1 Validation samples: {validation_generator_t1.samples}")
print(f"✓ T1 Test samples: {test_generator_t1.samples}")
print(f"✓ T1 Class indices: {train_generator_t1.class_indices}")
print()

# Sanity check: class order should match baseline (no comparisons are valid)
assert train_generator.class_indices == train_generator_t1.class_indices, \
    "Class index mapping differs between baseline and T1 generators."
print(f"✓ Class index mapping matches baseline.")

```

## 6. GitHub Repository

[https://github.com/SSPrajwala/BrainTumorDetection\\_Internship](https://github.com/SSPrajwala/BrainTumorDetection_Internship)